

Atelier Lens API 명세서

MVP v3.0 - 개발 확정본

최종 확정: 2026-08-13

기준 자료: 발품단_기획서.docx , 기능별_시나리오.pdf , 팀 백엔드 의사결정

본 문서는 프론트엔드-백엔드 연동과 백엔드 구현의 단일 기준 문서다. 기획서의 핵심 흐름인 **제품 품번 촬영 → 가상 피팅 → 규칙 기반 스타일 추천 → 코디/장바구니 → 모의결제 → 사후관리**를 기능별 시나리오에 맞춰 API로 구체화한다.

0. 최종 확정사항

항목	확정 내용
Backend	Python + FastAPI
Database	PostgreSQL
ORM / Migration	SQLAlchemy + Alembic
API Prefix	/api/v1
ID	UUID
DB / JSON naming	camelCase
Datetime	KST, ISO 8601 +09:00
인증	게스트 토큰 + 회원 JWT
Access Token	2시간
Refresh Token	14일, rotation 적용 권장
Password	bcrypt
Social Login	Google, Kakao
동일 이메일	LOCAL / GOOGLE / KAKAO를 별도 계정으로 생성
QR	선택적 진입 컨텍스트. opaque code를 서버가 매장/캠페인에 매핑
제품 인식	QR/바코드 미사용. 품번 사진을 서버 OCR로 인식
OCR	PaddleOCR PP-OCrv5 + OpenCV 전처리, 필요 시 2차 전처리 재검증
추천	Rule-based, RecommendationService 인터페이스로 AI 확장 가능
가상 피팅	TryOnService 인터페이스 + MVP MockTryOnProvider
비동기	FastAPI BackgroundTasks + PostgreSQL Job 상태 저장
AI HTTP Timeout	60초
결제	PaymentProvider + MockPaymentProvider, 실제 PG 확장 가능
파일	별도 서버 경로에 저장, DB에는 경로/메타데이터만 저장

항목	확정 내용
개인 파일 접근	인증된 GET /files/{fileId} API
GuestSession	당일 23:59:59 KST 만료
개인 원본 사진	1시간 만료
Try-on 임시 결과	3시간 만료, 회원의 명시적 저장 시 영구화
Job	24시간 보관 후 정리
최대 이미지	품번/피팅/아바타/사후관리 사진 20MB
최대 영상	사후관리 MP4/MOV 100MB
기존 보유 제품	시리얼 등록 지원
사후관리	구매 제품 + 기존 등록 제품 모두 지원, AI 진단/수리예약 포함

1. 공통 규약

1.1 Base URL / Content-Type

```
https://api.example.com/api/v1
```

- 기본: application/json; charset=utf-8
- 파일 업로드: multipart/form-data
- 날짜/시간 응답: 2026-08-13T23:30:00+09:00
- DB와 JSON 필드명: camelCase

1.2 인증 레벨

```
Authorization: Bearer <token>
```

권한	의미
PUBLIC	인증 불필요
GUEST	게스트 토큰 또는 회원 JWT
MEMBER	회원 JWT 필수

1.3 공통 성공 응답

```
{
  "success": true,
  "data": {},
  "error": null,
  "meta": {
    "requestId": "req_8a79...",
    "pagination": null
  }
}
```

1.4 공통 실패 응답

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "입력값을 확인해주세요.",
    "details": {"field": "heightCm"}
  },
  "meta": {"requestId": "req_8a79..."}
}
```

1.5 HTTP 상태 코드

HTTP	사용 기준
200	조회/수정/삭제 성공
201	동기 리소스 생성 성공
202	Background Job 접수 성공
400	잘못된 요청 형식
401	토큰 없음/만료/로그인 실패
403	권한 부족 또는 소유권 불일치
404	리소스 또는 품번/시리얼 없음
409	중복/재고/예약 충돌
422	입력값·파일·OCR 처리 검증 실패
429	게스트 사용 제한 초과/로그인 잠금
500	서버 내부 오류
503	외부 Provider 또는 AI 이용 불가

1.6 공통 에러 코드

code	HTTP	의미
UNAUTHORIZED	401	인증 실패
TOKEN_EXPIRED	401	토큰 만료
FORBIDDEN	403	권한/소유권 부족
VALIDATION_ERROR	422	입력값 오류
NOT_FOUND	404	일반 리소스 없음
CONFLICT	409	상태 충돌
GUEST_SESSION_EXPIRED	401	게스트 세션 만료
GUEST_LIMIT_EXCEEDED	429	게스트 제한 초과
FILE_TOO_LARGE	422	파일 최대 용량 초과

code	HTTP	의미
UNSUPPORTED_FILE_TYPE	422	허용하지 않은 파일
AI_TIMEOUT	503	AI 처리 60초 초과
AI_UNAVAILABLE	503	AI Provider 이용 불가
GENERATION_FAILED	200	Job 내부 생성 실패
INTERNAL_ERROR	500	서버 내부 오류

1.7 커서 페이지네이션

```
?cursor=<opaqueCursor>&limit=20
```

- 기본 limit=20, 최대 50
- 목록 응답은 meta.pagination = {nextCursor, hasNext, limit}

2. 게스트 세션 / QR 컨텍스트

QR은 제품 인식에 사용하지 않는다. QR에는 짧은 opaque code만 저장하며 서버가 Store / Campaign 에 매핑한다. QR 없이도 게스트 진입 가능하다.

POST /guest-sessions [PUBLIC]

Request - 일반 게스트

```
{}
```

Request - QR 진입

```
{"qrCode": "a7B9x2"}
```

Response 201

```
{
  "success": true,
  "data": {
    "guestToken": "...",
    "guestSessionId": "uuid",
    "store": {"storeId": "uuid", "name": "MCM 플래그십"},
    "campaign": {"campaignId": "uuid", "name": "Atelier Lens Demo"},
    "expiresAt": "2026-08-13T23:59:59+09:00"
  },
  "error": null,
  "meta": {"requestId": "req_..."}
}
```

- qrCode 가 없으면 store, campaign 은 null 가능
- QR 오류: QR_INVALID (404), QR_EXPIRED (422)
- 세션에는 게스트의 heightCm, weightKg, gender 를 선택적으로 유지할 수 있다.

3. 인증 / 회원

POST /auth/signup [PUBLIC]

```
{
  "email": "user@example.com",
  "password": "*****",
  "name": "홍길동",
  "phone": "010-1234-5678"
}
```

- 필수: email, password, name
- 선택: phone
- 비밀번호: 최소 8자, 대문자/숫자/특수문자 규칙
- LOCAL 이메일 중복만 EMAIL_ALREADY_EXISTS (409)

Response 201: userId 반환. 회원가입 후 화면 흐름은 로그인 화면으로 이동한다.

POST /auth/login [PUBLIC]

```
{"email": "user@example.com", "password": "*****"}
```

Response 200

```
{
  "success": true,
  "data": {
    "userId": "uuid",
    "accessToken": "...",
    "refreshToken": "...",
    "accessTokenExpiresIn": 7200,
    "refreshTokenExpiresIn": 1209600
  },
  "error": null,
  "meta": {"requestId": "req_..."}
}
```

- 5회 연속 로그인 실패 시 일시 잠금: LOGIN_TEMPORARILY_LOCKED (429)

POST /auth/social [PUBLIC]

Google/Kakao Authorization Code를 서버가 교환·검증한 뒤 서비스 JWT를 발급한다.

```
{
  "provider": "google",
  "authorizationCode": "...",
  "redirectUri": "https://frontend.example.com/oauth/callback"
}
```

provider: google | kakao

같은 이메일이라도 LOCAL, GOOGLE, KAKAO 는 서로 다른 User 계정이다. 이메일만으로 계정을 자동 병합하지 않는다.

POST /auth/refresh [PUBLIC]

```
{"refreshToken": "..."} 
```

- Refresh Token 14일
- 재발급 시 rotation 권장: 기존 토큰 폐기 후 새 refresh token 발급

POST /auth/logout [MEMBER]

```
{"refreshToken": "..."} 
```

서버의 refresh token hash를 폐기한다.

POST /auth/claim [MEMBER]

게스트 세션의 영구화 가능한 데이터를 로그인 회원에게 이관한다.

```
{"guestToken": "..."} 
```

이관 대상

- 키/몸무게/성별 아바타 파라미터
- 최근 본 상품
- 명시적으로 저장 요청한 Try-on 결과
- 현재 코디 컨텍스트(있는 경우)

이관 제외

- 원본 개인 사진
- 만료된 Job
- 임시 AI 중간 파일

4. 내 계정 / 아바타

GET /me [MEMBER]

회원 프로필과 아바타 존재 여부를 반환한다.

PATCH /me [MEMBER]

```
{"name": "홍길동", "phone": "010-0000-0000"} 
```

PATCH /me/password [MEMBER]

```
{"currentPassword": "...", "newPassword": "..."} 
```

소셜 로그인 전용 계정은 비밀번호 변경 불가: PASSWORD_AUTH_NOT_AVAILABLE (409)

POST /me/avatar [MEMBER]

```
{"heightCm": 172, "weightKg": 65, "gender": "female"}
```

- gender: female | male | neutral
- 키 100~230cm, 몸무게 30~200kg
- 아바타 이미지 생성은 MVP에서는 Mock 가능

GET /me/avatar [MEMBER]

현재 아바타 파라미터/미리보기 조회.

PUT /me/avatar [MEMBER]

키/몸무게/성별을 수정하고 아바타를 갱신한다.

PUT /guest-sessions/me/avatar-parameters [GUEST]

게스트가 신체정보를 입력한 경우 세션에만 저장한다. 회원 저장은 /auth/claim 이후 /me/avatar 로 영구화한다.

```
{"heightCm": 172, "weightKg": 65, "gender": "female"}
```

5. 제품 품번 사진 OCR / 상품

제품 품번은 카메라 사진으로만 인식한다. 프론트는 사진만 전송하며 OCR·정규화·상품 조회는 백엔드가 수행한다.

POST /product-recognitions [GUEST]

multipart/form-data

필드	타입	필수	제한
image	file	O	JPEG/PNG/WEBP, 20MB

처리 흐름

```
image
→ 파일 signature 검증
→ OpenCV 전처리
→ PaddleOCR PP-OCRv5 1차 인식
→ 품번 후보 정규화
→ Product DB 정확 일치 조회
→ 미탐지/저신뢰 시 다른 전처리로 2차 OCR
→ 최종 후보 선택
```

실제 품번 포맷은 아직 미확정이므로 정규식은 설정값/전략으로 분리한다. 초기에는 영문·숫자·- 등 일반적 품번 문자를 허용하고, 실제 데이터 확보 후 제한을 강화한다.

Response 200

```
{
  "success": true,
  "data": {
    "recognizedCode": "MCM-ABC123",
    "confidence": 0.96,
    "product": {
```

```

    "productId": "uuid",
    "productCode": "MCM-ABC123",
    "name": "MCM Backpack",
    "thumbnailFileId": "uuid",
    "basePrice": 890000,
    "currency": "KRW"
  }
},
"error": null,
"meta": {"requestId": "req_..."}
}

```

전용 에러

- `PRODUCT_CODE_NOT_DETECTED` (422)
- `PRODUCT_CODE_AMBIGUOUS` (422)
- `PRODUCT_NOT_FOUND` (404)

OCR 원본 사진은 상품 식별 후 영구 보관하지 않는다.

GET `/products/{productId}` [GUEST]

상품 상세 + variants + 이미지 + 태그를 반환한다.

GET `/products/{productId}/variants` [GUEST]

색상/사이즈/SKU/현재 가격/재고 조회.

GET `/recent-products` [GUEST]

현재 게스트 세션 또는 회원의 최근 스캔/조회 상품을 최신순으로 반환한다.

6. 규칙 기반 추천

GET `/products/{productId}/recommendations` [GUEST]

Query

```
?limit=10
```

- 회원: 최대 20개
- 게스트: 서버가 최대 **3개**로 강제 제한

MVP 추천 점수 예시

- `category` 상호 보완
- `styleTags` 일치
- `colorTags` 조화
- `seasonTags` 일치
- 동일 상품 제외 / 비활성 상품 제외 / 재고 없는 Variant 제외

구현은 `RecommendationService` 인터페이스 아래 `RuleBasedRecommendationService`를 사용한다. 추후 AI 추천으로 교체해도 endpoint 계약은 유지한다.

7. Job

GET /jobs/{jobId} [GUEST]

```
{
  "success": true,
  "data": {
    "jobId": "uuid",
    "type": "photoTryOn",
    "status": "processing",
    "progress": 45,
    "result": null,
    "error": null,
    "expiresAt": "2026-08-14T23:30:00+09:00"
  },
  "error": null,
  "meta": {"requestId": "req_..."}
}
```

status: pending | processing | succeeded | failed

- Job 상태는 PostgreSQL에 저장한다.
- FastAPI BackgroundTasks 로 단일 서버에서 수행한다.
- 서버 재시작 시 pending/processing 잔존 Job을 failed + SERVER_RESTARTED 로 정리할 수 있어야 한다.
- Job은 24시간 후 cleanup 대상이다.

8. 아바타 가상 착용

POST /avatar-try-ons [GUEST]

```
{
  "productId": "uuid",
  "variantId": "uuid",
  "heightCm": 172,
  "weightKg": 65,
  "gender": "female"
}
```

- 회원은 /me/avatar 값을 생략 가능
- 게스트는 신체정보가 없으면 표준 아바타 사용
- MVP Provider: MockTryOnProvider

Response 202

```
{"success": true, "data": {"jobId": "uuid", "type": "avatarTryOn"}, "error": null, "meta": {"requestId": "req_..."}}
```

완료 Job result:

```
{"tryOnId": "uuid", "resultFileId": "uuid", "expiresAt": "2026-08-14T02:30:00+09:00"}
```

9. 내 사진 가상 착용

POST /try-ons [GUEST]

multipart/form-data

필드	타입	필수	설명
photo	file	O	JPEG/PNG/WEBP, 20MB
productId	UUID	조건부	제품 단일 피팅
savedCoordiId	UUID	조건부	저장 코디 전체 피팅
variantId	UUID	X	색상/사이즈 variant
scope	enum	O	productOnly fullCoordi

- 게스트 사진 업로드/피팅 시도는 세션당 최대 3회, 서버에서 카운트
- 원본 개인 사진 TTL: 1시간
- 합성 결과 TTL: 3시간
- PERSON_NOT_DETECTED, INVALID_IMAGE 등 Provider 에러를 공통 에러로 매핑

POST /try-ons/{tryOnId}/save [MEMBER]

명시적인 [결과 저장] 버튼에서 호출한다.

- 소유권 확인 후 임시 결과를 회원 영구 파일로 전환
- expiresAt = null, savedAt 기록
- 원본 개인 사진은 저장하지 않고 TTL에 따라 삭제

GET /me/try-ons [MEMBER]

저장한 Try-on 결과 목록. 현재 화면에 직접 노출하지 않더라도 저장 결과 관리/향후 확장용으로 유지한다.

DELETE /me/try-ons/{tryOnId} [MEMBER]

저장 결과 삭제.

10. 저장한 코디

POST /me/coordis [MEMBER]

```
{
  "name": "내 코디",
  "items": [
    {"productId": "uuid", "variantId": "uuid"},
    {"productId": "uuid", "variantId": "uuid"}
  ]
}
```

GET /me/coordis [MEMBER]

저장 코디 목록.

GET `/me/coords/{savedCoordId}` [MEMBER]

코디 상세.

PATCH `/me/coords/{savedCoordId}` [MEMBER]

아이템 추가/삭제/variant 변경/이름 수정.

DELETE `/me/coords/{savedCoordId}` [MEMBER]

Soft Delete.

11. 장바구니

장바구니는 회원 전용이다. 게스트가 담기를 누르면 로그인 후 요청한다.

POST `/cart/items` [MEMBER]

```
{"variantId": "uuid", "quantity": 1}
```

서버는 현재 상품 상태, 실제 가격, 재고를 기준으로 검증한다.

GET `/cart` [MEMBER]

장바구니 목록/총액.

PATCH `/cart/items/{cartItemId}` [MEMBER]

```
{"quantity": 2}
```

또는 variant 변경.

DELETE `/cart/items/{cartItemId}` [MEMBER]

Hard Delete 가능.

12. 주문 / 모의결제

POST `/orders` [MEMBER]

선택한 장바구니 항목을 주문하고 Mock 결제를 수행한다.

```
{
  "cartItemIds": ["uuid", "uuid"],
  "paymentMethod": "mock"
}
```

서버 처리 순서

1. CartItem 소유권 검증
2. ProductVariant 활성/재고 검증
3. DB의 현재 가격으로 금액 재계산
4. Order `PENDING` 생성
5. `PaymentProvider.pay()` 호출

6. MVP는 `MockPaymentProvider` 가 성공/실패 시나리오 반환
7. 성공: `Payment.SUCCESS`, `Order.PAID`, 재고 차감, 선택 `CartItem` 삭제
8. 구매 상품을 `RegisteredProduct(source=PURCHASE)` 로 자동 등록 가능

Response 201

```
{
  "success": true,
  "data": {
    "orderId": "uuid",
    "orderStatus": "paid",
    "paymentStatus": "success",
    "paidAmount": 1780000,
    "paidAt": "2026-08-13T23:40:00+09:00"
  },
  "error": null,
  "meta": {"requestId": "req_..."}
}
```

`Order.status`: pending | paid | failed | cancelled

`Payment.status`: ready | success | failed | cancelled | refunded

GET /me/orders [MEMBER]

결제내역 목록.

GET /me/orders/{orderId} [MEMBER]

주문/결제 상세와 주문 당시의 상품명, 가격, variant snapshot을 반환한다.

13. 보유 제품 / 사후관리

구매 제품과 기존 보유 제품 모두 `RegisteredProduct` 로 통합 관리한다.

POST /me/products [MEMBER]

기존 보유 제품을 시리얼로 등록한다.

```
{
  "serialNumber": "MCM-2024-XXXX",
  "purchaseDate": "2025-01-02",
  "nickname": "내 백팩"
}
```

- 실제 시리얼 포맷은 데이터 확보 후 강화
- `SERIAL_NOT_FOUND` (404), `ALREADY_REGISTERED` (409)
- `source = manual`

GET /me/products [MEMBER]

`source=purchase|manual` 을 포함한 내 제품 목록.

GET /me/products/{registrationId} [MEMBER]

제품 정보, 등록 출처,케어정보, 최근 진단/예약 요약.

GET /me/products/{registrationId}/care-guide [MEMBER]

제품 재질/카테고리별 기본 관리법, 주의사항, AS 정보.

14. AI 손상 진단

POST /diagnoses [MEMBER]

multipart/form-data, Background Job.

필드	타입	필수	제한
registrationId	UUID	O	본인 등록 제품
files	file[]	O	최대 5개
사진	JPEG/PNG/WEBP	-	파일당 20MB
영상	MP4/MOV	-	파일당 100MB

Response 202: jobId, type=diagnosis

MVP에서는 실제 AI 진단 모델이 준비되지 않은 경우 MockDiagnosisProvider를 사용해 인터페이스와 전체 흐름을 완성할 수 있다.

GET /diagnoses/{diagnosisId} [MEMBER]

```
{
  "success": true,
  "data": {
    "diagnosisId": "uuid",
    "registrationId": "uuid",
    "repairNeeded": true,
    "summary": "모서리 마모가 확인됩니다.",
    "damages": [
      {
        "category": "abrasion",
        "severity": 2,
        "confidence": 0.88,
        "location": "corner",
        "boundingBox": null
      }
    ],
    "createdAt": "2026-08-13T23:45:00+09:00"
  },
  "error": null,
  "meta": {"requestId": "req_..."}
}
```

초기 category: scratch | stain | tear | hardwareDamage | discoloration | deformation | abrasion

GET /diagnoses/{diagnosisId}/care-guide [MEMBER]

진단 결과를 반영한 홈케어 가이드.

15. 수리 예약 / 매장

GET /stores [MEMBER]

```
?date=2026-08-20&cursor=...&limit=20
```

매장 정보 및 예약 가능 슬롯을 반환한다.

POST /repair-reservations [MEMBER]

```
{
  "storeId": "uuid",
  "diagnosisId": "uuid",
  "slot": "2026-08-20T14:00:00+09:00",
  "memo": "지퍼 상태 확인 요청"
}
```

동일 슬롯 동시 예약 충돌: SLOT_UNAVAILABLE (409)

GET /me/repair-reservations [MEMBER]

내 수리예약 목록.

PATCH /me/repair-reservations/{reservationId} [MEMBER]

MVP에서는 예약 취소만 지원.

```
{"status": "cancelled"}
```

16. 파일 API / 스토리지

실제 파일은 DB BLOB으로 저장하지 않는다.

```
/data/atelier-lens/
products/
avatars/
uploads/guests/
uploads/members/
tryOn/
diagnosis/
temporary/
```

DB FileMetadata 에는 path, ownerType, ownerId, contentType, size, expiresAt, createdAt 등을 저장한다.

GET /files/{fileId} [GUEST]

- 개인 파일은 요청 토큰의 소유권을 검증한 뒤 FileResponse /stream으로 반환
- guest 파일은 해당 guest session만 접근
- member 파일은 해당 user만 접근
- 상품 공개 이미지는 별도의 public 경로 또는 visibility=public 정책 사용 가능
- 원본 개인 사진을 static URL로 직접 노출하지 않는다.

파일 포맷 / 용량

용도	포맷	최대
품번 OCR	JPEG, PNG, WEBP	20MB
개인 Try-on	JPEG, PNG, WEBP	20MB
아바타	JPEG, PNG, WEBP	20MB
사후관리 사진	JPEG, PNG, WEBP	20MB
사후관리 영상	MP4, MOV	100MB

검증은 extension + Content-Type + file signature 를 조합한다.

17. TTL / Cleanup

대상	정책
GuestSession	생성 당일 23:59:59 KST
게스트 키/몸무게/성별	GuestSession과 동일
개인 원본 Try-on 사진	업로드 후 1시간
임시 Try-on 결과	생성 후 3시간
명시적으로 저장한 회원 Try-on	영구(삭제 시까지)
Job	24시간
품번 OCR 이미지	인식 완료 후 삭제

CleanupService 는 만료 파일/DB row를 정리한다. 단일 서버 MVP에서는 주기 실행 루틴으로 구현하고, 추후 worker/scheduler로 교체 가능하게 서비스 계층을 분리한다.

18. Provider / Service Interface 계약

특정 외부 SDK/모델의 Request/Response 객체가 Router/도메인까지 퍼지지 않도록 인터페이스를 고정한다.

OCR

```
OcrService
recognizeProductCode(imagePath) -> OcrResult

PaddleOcrService implements OcrService
```

OcrResult: rawTexts, candidates, selectedText, confidence, boundingBoxes, processingTimeMs

Try-on

```
TryOnService
generate(input) -> TryOnResult
```

```
MockTryOnProvider implements TryOnService
FutureRealTryOnProvider implements TryOnService
```

```
TryOnResult : resultFileId, status, confidence?, provider, processingTimeMs
```

Recommendation

```
RecommendationService
  recommend(productId, limit, context) -> ProductRecommendation[]

RuleBasedRecommendationService implements RecommendationService
```

Payment

```
PaymentProvider
  pay(order, paymentRequest) -> PaymentResult

MockPaymentProvider implements PaymentProvider
FuturePgProvider implements PaymentProvider
```

19. 주요 화면 ↔ API 매핑

기능 시나리오	주요 API
SP-00 앱 시작	로컬 토큰 확인 → 필요 시 /auth/refresh
LG-00 로그인	POST /auth/login
LG-01 회원가입	POST /auth/signup
LG-02 소셜로그인	POST /auth/social
LG-03 게스트	POST /guest-sessions
MN-01 품번 스캔	POST /product-recognitions
MN-02 아바타로 보기	POST /avatar-try-ons → /jobs/{id}
MN-03 내 사진으로 보기	POST /try-ons → /jobs/{id}
코디 추천	GET /products/{id}/recommendations
코디 저장	POST /me/coordis
최근 본 상품	GET /recent-products
CT-01 장바구니	/cart/items, /cart
CT-02 구매하기	POST /orders
MY-02 저장 코디	GET /me/coordis
MY-04 결제 내역	GET /me/orders
MY-05~07 사후관리	/me/products, /care-guide, /diagnoses
MY-08 계정	GET/PATCH /me, /me/password
MY-09~10 아바타	GET/POST/PUT /me/avatar

20. 엔드포인트 요약

#	Method	Path	권한	Async
1	POST	/guest-sessions	PUBLIC	
2	POST	/auth/signup	PUBLIC	
3	POST	/auth/login	PUBLIC	
4	POST	/auth/social	PUBLIC	
5	POST	/auth/refresh	PUBLIC	
6	POST	/auth/logout	MEMBER	
7	POST	/auth/claim	MEMBER	
8	GET	/me	MEMBER	
9	PATCH	/me	MEMBER	
10	PATCH	/me/password	MEMBER	
11	POST	/me/avatar	MEMBER	
12	GET	/me/avatar	MEMBER	
13	PUT	/me/avatar	MEMBER	
14	PUT	/guest-sessions/me/avatar-parameters	GUEST	
15	POST	/product-recognitions	GUEST	
16	GET	/products/{productId}	GUEST	
17	GET	/products/{productId}/variants	GUEST	
18	GET	/recent-products	GUEST	
19	GET	/products/{productId}/recommendations	GUEST	
20	GET	/jobs/{jobId}	GUEST	polling
21	POST	/avatar-try-ons	GUEST	O
22	POST	/try-ons	GUEST	O
23	POST	/try-ons/{tryOnId}/save	MEMBER	
24	GET	/me/try-ons	MEMBER	
25	DELETE	/me/try-ons/{tryOnId}	MEMBER	
26	POST	/me/coordis	MEMBER	
27	GET	/me/coordis	MEMBER	
28	GET	/me/coordis/{savedCoordiId}	MEMBER	
29	PATCH	/me/coordis/{savedCoordiId}	MEMBER	
30	DELETE	/me/coordis/{savedCoordiId}	MEMBER	

#	Method	Path	권한	Async
31	POST	/cart/items	MEMBER	
32	GET	/cart	MEMBER	
33	PATCH	/cart/items/{cartItemId}	MEMBER	
34	DELETE	/cart/items/{cartItemId}	MEMBER	
35	POST	/orders	MEMBER	
36	GET	/me/orders	MEMBER	
37	GET	/me/orders/{orderId}	MEMBER	
38	POST	/me/products	MEMBER	
39	GET	/me/products	MEMBER	
40	GET	/me/products/{registrationId}	MEMBER	
41	GET	/me/products/{registrationId}/care-guide	MEMBER	
42	POST	/diagnoses	MEMBER	O
43	GET	/diagnoses/{diagnosisId}	MEMBER	
44	GET	/diagnoses/{diagnosisId}/care-guide	MEMBER	
45	GET	/stores	MEMBER	
46	POST	/repair-reservations	MEMBER	
47	GET	/me/repair-reservations	MEMBER	
48	PATCH	/me/repair-reservations/{reservationId}	MEMBER	
49	GET	/files/{fileId}	GUEST	

21. 아직 데이터 확보 후 조정할 값

아래 항목은 API 구조를 바꾸지 않고 설정/검증 규칙만 강화한다.

1. 실제 MCM 품번 패턴/길이/허용 문자
2. 실제 제품/Variant/시리얼 seed 데이터
3. 실제 Virtual Try-on 모델/API 연결 - 현재 `MockTryOnProvider`
4. 실제 AI 손상 진단 모델 연결 - 필요 시 `MockDiagnosisProvider` 에서 교체
5. 실제 PG 연결 - 현재 `MockPaymentProvider`

이 항목들은 Provider/Service Interface 뒤에 숨기므로 프론트 API 계약과 주요 DB 구조는 유지한다.